



Prepared by group 14

Smart Voice Assistant

Python Project



Team Members

Pachigulla Joshika	AP23110011569
Kunam Day Harika	AP23110011607
Aasritha Daisy Lam	AP23110011610
Sakala Bhagya Sri	AP23110011618



Introduction



Our project is a python-based **voice assistant** designed to perform tasks using voice commands.

Named '**Sophia**', the assistant integrates voice recognition, text-to-speech, and automation features.

It is useful for daily productivity and hands-free operation of basic tasks.

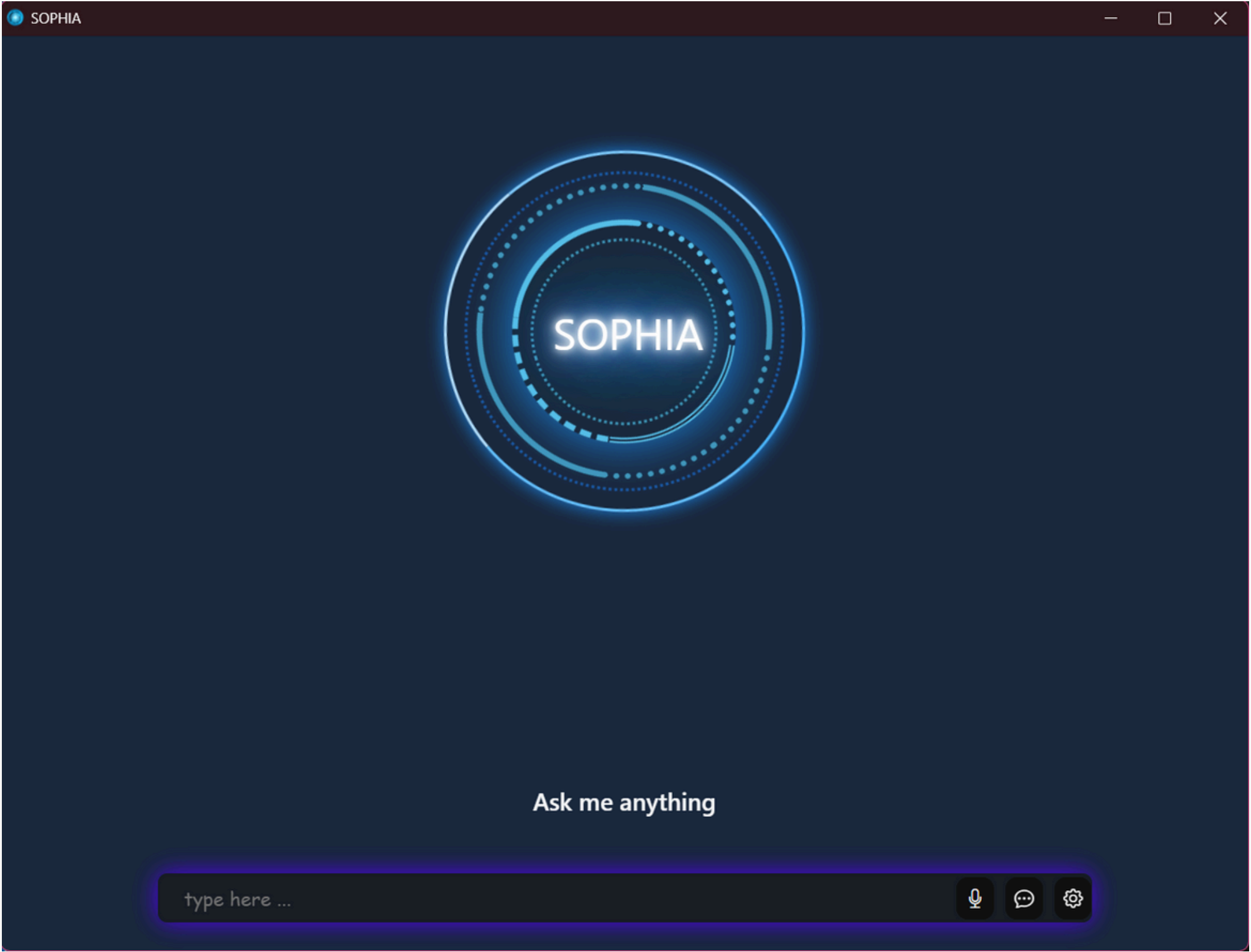


Technologies Used

- Python
- Eel (for connecting Python with frontend)
- pyttsx3 (Text-to-Speech)
- SpeechRecognition (Speech-to-Text)
- PyAutoGUI, Requests, Wikipedia, pywhatkit, etc.



Front End

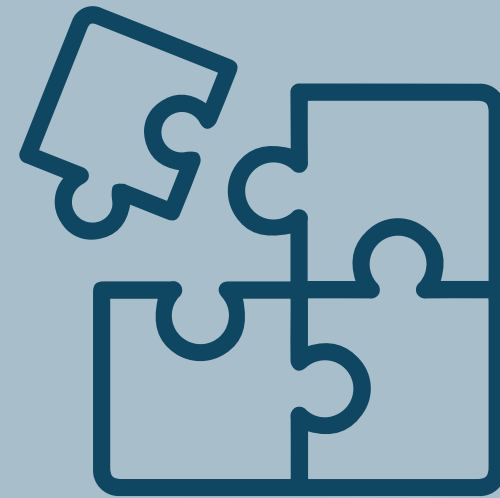


System Architecture



Voice Input

SpeechRecognition
captures user input



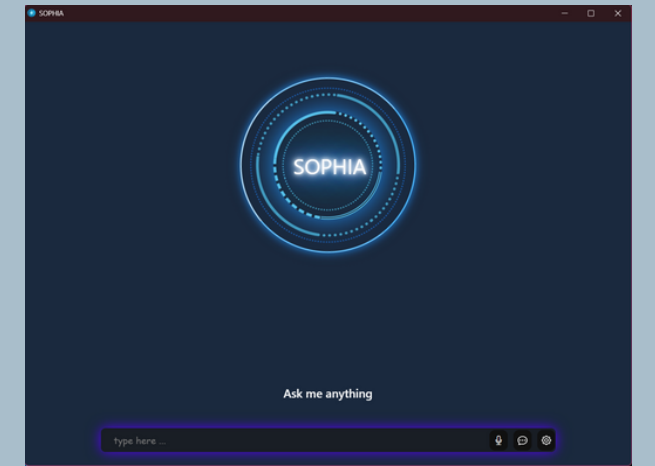
Command Processing

Keywords parsed
and directed



Action Execution

Features executed
via Python modules



Frontend

Connected using Eel
and displayed in
browser

Key Features

- Open applications and websites
 - Play YouTube videos
 - Fetch latest news and stock prices
 - Take notes and screenshots
 - Search Google/Wikipedia
 - Set alarms and timers
-

Methodology

How Commands Work

- User speaks a command (e.g., 'play music on YouTube')
- Speech is recognized and converted to text
- Text is analyzed to identify the required action
- Corresponding function is called (e.g., PlayYoutube)

Interesting Code Logic

- Dynamic command routing in `command.py`
- Threading used for alarms without blocking UI
- Fallback logic for app/URL handling in `openCommand()`
- Error handling in Wikipedia and News fetching

Frontend Integration

- Eel used to bind Python backend with a web frontend
- Voice responses and messages displayed via HTML/JS
- Browser launched as a desktop app using Chrome app mode

EXAMPLE USECASE

- User says: 'Set a timer for 2 minutes'
- Assistant processes and waits 2 minutes
- Then says: 'Time's up!' via speaker and browser UI

FUTURE SCOPE

- **Better understanding:** Improve how the assistant understands what we say by using smarter language tools.
- **Voice lock:** Add a feature to recognize the user's voice for security.
- **Always listening mode:** Make it listen for a wake word like "Hey Assistant" without clicking a button.

Conclusion



In this project, we built a voice assistant that can understand our voice, perform different tasks like opening apps, searching online, reading news, setting alarms, and more. It talks back to us using text-to-speech and shows messages on a simple interface. The assistant works well, and the project shows how we can use Python to control a computer using voice commands.





Thank you

